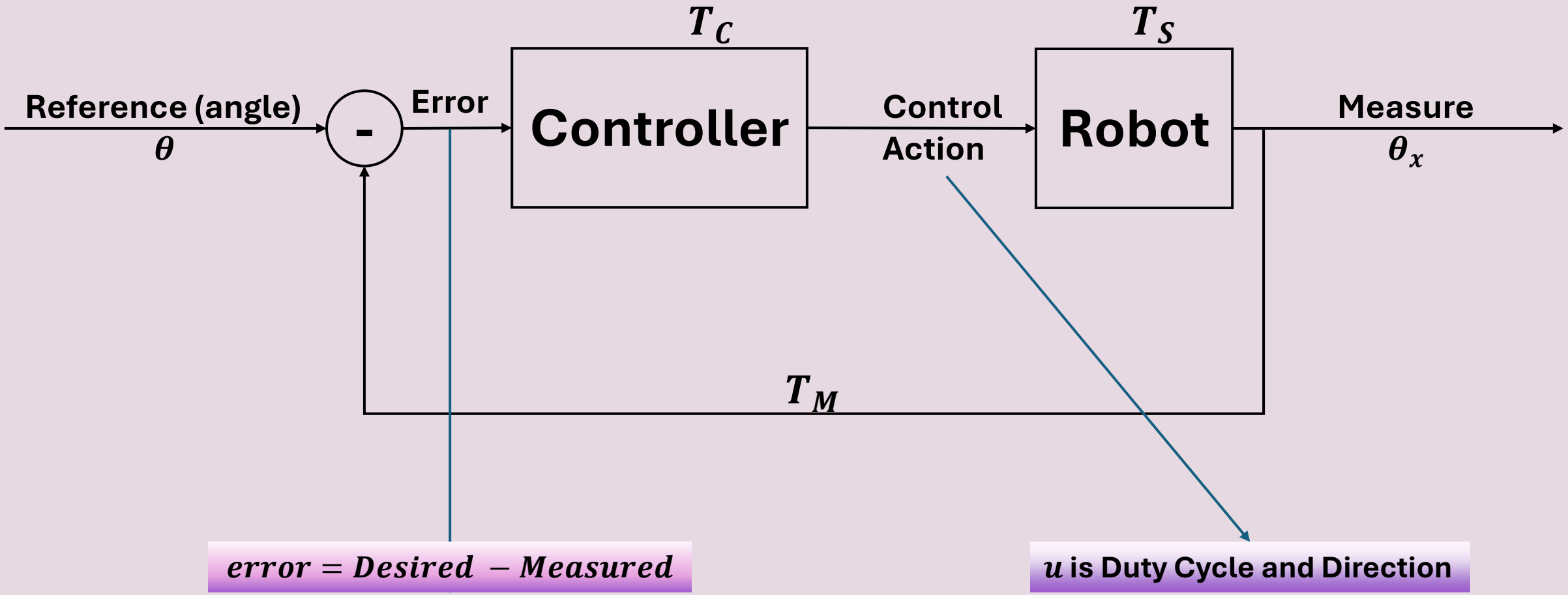
The Control Loop

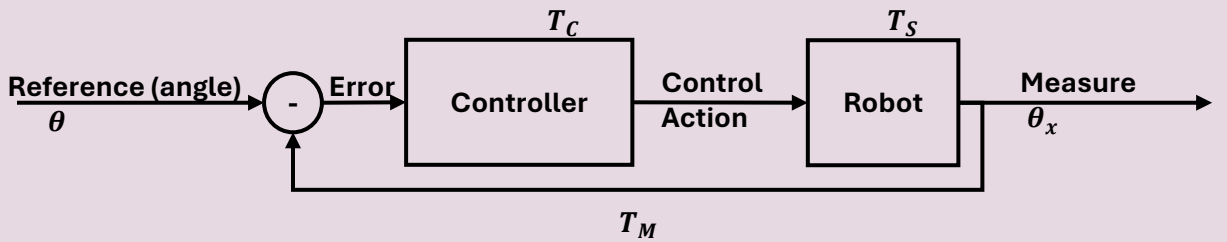
The Control System and Control Loop

Loop Time: $dt > T_c + T_s + T_m$



Control Loop Design 1

Using Loop Delay



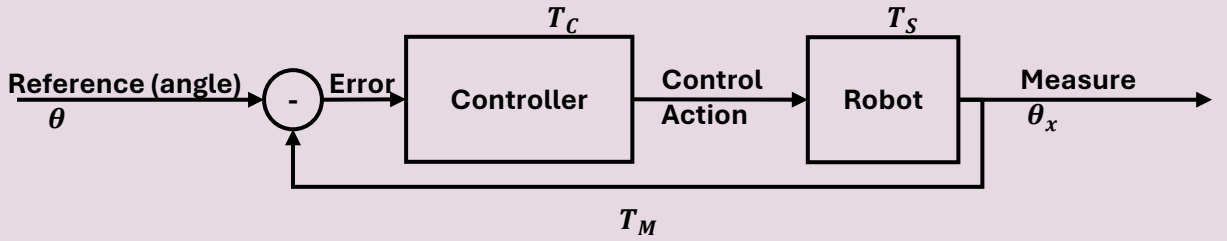

```
unsigned long lastLoopTime = 0;
TARGET_DT_US = 0.01
void loop()
{
    unsigned long currentTime = micros();
    unsigned long elapsedTime = currentTime - lastLoopTime;

    Measure()
    Control(TARGET_DT_US)

    if (elapsedTime >= TARGET_DT_US) {
        lastLoopTime = currentTime; \\Warning
    }
    else {
        while ((micros() - lastLoopTime) < TARGET_DT_US) {
        }
    lastLoopTime += TARGET_DT_US;
    }
}
```

Control Loop Design 1

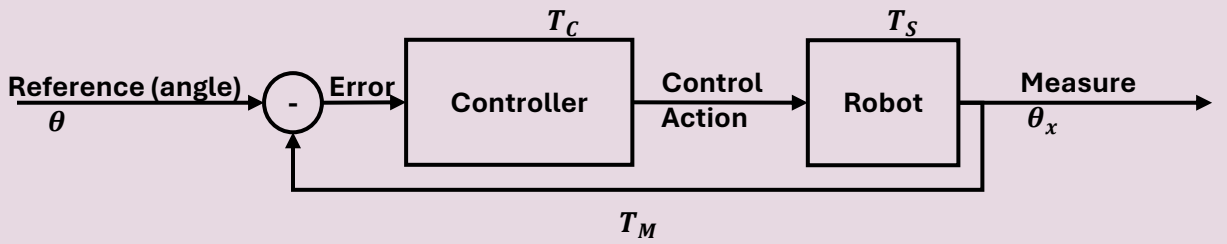
Using Timer




```
hw_timer_t *controlTimer = NULL;
void IRAM_ATTR onTimer()
{
        measure(); \\ T_m + T_s
        control(dt); \\ T_c
}
void measure() {
}
void control()
{
}
void setup()
{
    controlTimer = timerBegin(1000000);
    timerAttachInterrupt(controlTimer, &onTimer, true);
    timerAlarmWrite(controlTimer, 1000, true);
    timerAlarmEnable(controlTimer);
}
void loop() {}
```

Control Loop Design 1

Using RTOS



*Using **FreeRTOS** tasks is often preferred over hardware timer interrupts on the ESP32 for control loops. Instead of handling raw interrupts (ISRs), you run a dedicated **FreeRTOS** task that uses **vTaskDelayUntil().** This guarantees a precise period without cumulative time drift while keeping the CPU free for other tasks or background Wi-Fi/Bluetooth operations.*

THE PID

$$u(t) = \underbrace{K_p e(t)}_{P} + \underbrace{K_i \int_0^T e(t)\,dt}_{I} + \underbrace{K_d \frac{de(t)}{dt}}_{D}$$

e = Desired angle – Measured Angle

P = K_p * e

I_e += e*dt;
I = K_i * I_

D_e= (e-e_prev)/dt;
D = K_d * D_e

$$u = min(max((P + I + D), u_{min}), u_{max})$$

THE PID (with Clamp)

```
float K_p = 1.0;
float K_i = 0.5;
float K_d = 0.1;
float integral_error = 0.0;
float prev_error = 0.0;
const float dt = 0.01;
const float OUTPUT_MIN = -255.0;
const float OUTPUT_MAX = 255.0;
const float INTEGRAL_MIN = -200.0;
const float INTEGRAL_MAX = 200.0;

float computePID(float setpoint, float measured) {
        float e = setpoint - measured;
        integral_error += e * dt;
        integral_error = max(INTEGRAL_MIN,
min(integral_error, INTEGRAL_MAX));  \\Antiwindup
        float derivative = (e - prev_error) / dt;
        float output = (K_p * e) + (K_i * integral_error) +
(K_d * derivative);
output = max(OUTPUT_MIN, min(output,
OUTPUT_MAX));  \\Saturation
prev_error = e;  return output;}
```

THE PID (with Back Calculation)

```
float K_p = 1.0;
float K_i = 0.5;
float K_d = 0.1;
float K_bc = 1.0;
float integral_error = 0.0;
float prev_error = 0.0;
const float dt = 0.01;
const float OUTPUT_MIN = -255.0;
const float OUTPUT_MAX = 255.0;

float computePID(float setpoint, float measured)
{
    float e = setpoint - measured;
    float derivative = (e - prev_error) / dt;

    float unconstrained_output = (K_p * e) + (K_i *
integral_error) + (K_d * derivative);

    float output = max(OUTPUT_MIN,
min(unconstrained_output, OUTPUT_MAX));

    float saturation_error = output -
unconstrained_output;
    integral_error += (e * dt) + (K_bc *
saturation_error * dt);

    prev_error = e;
    return output;
}
```

THANK YOU